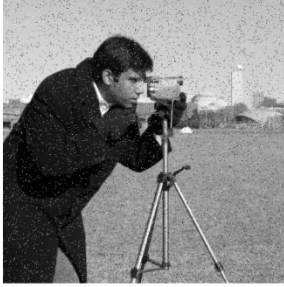


Digital Image Analysis Tutorial 5

```
1  """AIS31C T05 studio – The Filter Tournament (Tue 1 Sep, 2-3 PM).
2  INPUT: camera + two corruptions (Gaussian sigma 20; salt&pepper 4%), fixed seed.
3  OPERATION: run mean/Gaussian/median (5x5) on BOTH noises; fill the 3x2 PSNR table;
4  | then the parameter round: window 3/5/9 for the winner on each noise.
5  PARAMETERS: seed 5; window 5 default.
6  EXPECTED OUTPUT: PSNR table printed (bracket asserted) + expected_outputs/T05_panel.png.
7  INTERPRETATION: filters are hypotheses about the noise; match the hypothesis, win the cell.
8  """
9
10 import numpy as np, cv2, os, matplotlib
11 matplotlib.use("Agg"); import matplotlib.pyplot as plt
12 from skimage import data
13 HERE = os.path.dirname(os.path.abspath(_file_))
14 os.makedirs(f"{HERE}/expected_outputs", exist_ok=True)
15 rng = np.random.default_rng(5)
16 img = data.camera().astype(np.float64)
17 def psnr(x): return 10*np.log10(255**2/np.mean((img-x)**2))
18 gau = np.clip(img + rng.normal(0, 20, img.shape), 0, 255).astype(np.uint8)
19 sp = img.copy().astype(np.uint8); m = rng.random(img.shape); sp[m<0.02]=0; sp[m>0.98]=255
20
21 F = { "mean": lambda x, k: cv2.blur(x, (k, k)),
22       "gauss": lambda x, k: cv2.GaussianBlur(x, (k, k), 0),
23       "median": lambda x, k: cv2.medianBlur(x, k) }
24 print(f"noisy inputs: gaussian {psnr(gau):.1f} dB | salt&pepper {psnr(sp):.1f} dB")
25 table = {}
26 for fn, f in F.items():
27     a, b = psnr(f(gau, 5).astype(np.float64)), psnr(f(sp, 5).astype(np.float64))
28     Q table[fn] = (a, b); print(f" {fn:6s} 5x5: on-gaussian {a:.1f} dB | on-s&p {b:.1f} dB")
29 assert table["median"][1] > table["mean"][1] and table["median"][1] > table["gauss"][1]
30 best_g = max(table, key=lambda k: table[k][0])
31 print(f"[bracket] gaussian-noise winner: {best_g}; s&p winner: median (assert holds)")
32 for k in (3, 5, 9):
33     print(f" parameter round, median {k}x{k} on s&p: {psnr(cv2.medianBlur(sp,k).astype(np.float64)):.1f} dB")
34
35 fig, ax = plt.subplots(1, 4, figsize=(13, 3.6))
36 for a, im, t in zip(ax, [sp, F["mean"](sp,5), F["median"](sp,5), F["median"](sp,9)],
37                        [f"s&p input {psnr(sp):.1f} dB", f"mean 5x5 {table['mean'][1]:.1f} dB",
38                        f"median 5x5 {table['median'][1]:.1f} dB", f"median 9x9 {psnr(cv2.medianBlur(sp,9).astype(np.float64)):.1f} dB"]):
39     a.imshow(im, cmap="gray", vmin=0, vmax=255); a.set_title(t, fontsize=13); a.axis("off")
40 plt.tight_layout(); plt.savefig(f"{HERE}/expected_outputs/T05_panel.png", dpi=120); plt.close()
41 print("saved expected_outputs/T05_panel.png")
42
43 def support():
44     print("[SUPPORT] run only the s&p column: mean vs median at 5x5. Record both PSNRs and")
45     print(" write one line: what does the mean do to a black impulse that the median refuses to do?")
46
47 def extension():
48     bil = cv2.bilateralFilter(gau.astype(np.uint8), 9, 60, 10)
49     print(f"[EXTENSION] bilateral on gaussian noise: {psnr(bil.astype(np.float64)):.1f} dB – beat the 5x5 gaussian blur?")
50     print(" Tune (d, sigmaColor, sigmaSpace) for 5 minutes; report your best and the setting.")
51
52 if __name__ == "__main__":
53     import sys
54     if len(sys.argv) > 1 and sys.argv[1] == "support": support()
55     elif len(sys.argv) > 1 and sys.argv[1] == "extension": extension()
```

Output Images:

s&p input 18.8 dB



mean 5x5 25.5 dB



median 5x5 27.9 dB



median 9x9 24.8 dB

